KHULNA UNIVERSITY OF ENGINEERING & TECHNOLOGY (KUET)

Department of Electrical and Electronic Engineering

INDEPENDENT ENGINEERING PROJECT REPORT

Design, Hardware Architecture, and Discrete Multi-Zone Control of an Autonomous Line Follower Robot

A Technical Implementation of 8-Channel QTR-8RC Reflectance Sensing, Buck-Regulated Power Isolation, and 7-Zone Asymmetric Bang-Bang Navigation

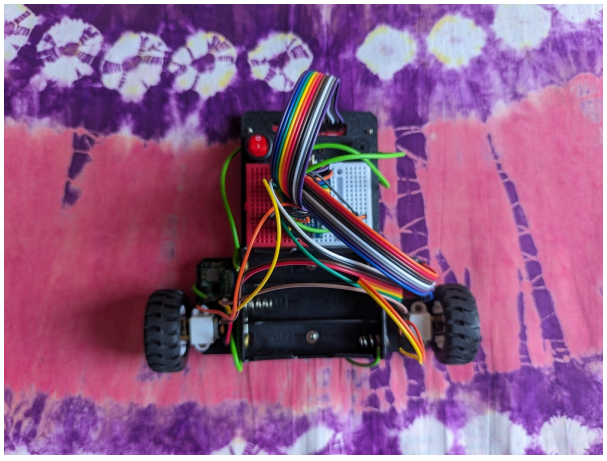


Figure 1: Top-down architectural overview of the completed 2WD autonomous line-following robot chassis.

Project Author:	Muhammad Mostafa Sajid Zeebran
Department:	Department of Electrical and Electronic Engineering (EEE)
University:	Khulna University of Engineering & Technology (KUET)

1. PROJECT OBJECTIVES

The primary objective of this personal engineering project is to design, fabricate, and validate an autonomous two-wheel-drive (2WD) line-following robot capable of tracking a high-contrast black trajectory across variable curvature geometries. Specifically, the project goals encompass: (1) interfacing an 8-channel RC infrared reflectance sensor array (`QTR-8RC`) with an Arduino Nano microcontroller for real-time spatial position estimation; (2) designing a transient-immune, dual-rail power distribution network utilizing a dedicated DC-DC step-down buck converter to prevent logic IC burnout; and (3) implementing a low-latency 7-zone discrete bang-bang control law with state-memory line loss recovery.

2. HARDWARE COMPONENTS & SPECIFICATIONS

Subsystem	Hardware Specification	MCU Pin Mapping	Functional Role
Microcontroller	Arduino Nano (ATmega328P, 16 MHz)	Red Mini-Breadboard	Sensor acquisition & PWM control
Reflectance Array	Pololu 8-Channel QTR-8RC Array	`D9, D8, D7, D6, D5, D4, D3, D2`	RC discharge optical line sensing
IR Emitter Control	QTR-8RC LEDON Pin	A3 (Digital HIGH)	Enables active IR LED emitter rail
Motor Driver IC	Dual H-Bridge Driver (TB6612FNG)	White Mini-Breadboard	Bidirectional current & PWM switching
Left Drive Channel	Motor Channel A (Left Gearmotor)	`PWMA: D10`, `AIN1: D12`, `AIN2: D13`	Left wheel torque & direction
Right Drive Channel	Motor Channel B (Right Gearmotor)	`PWMB: D11`, `BIN1: A0`, `BIN2: A1`	Right wheel torque & direction
Voltage Regulator	DC-DC Step-Down Buck Converter	5.0 V Regulated Logic Rail	Steps down battery voltage for MCU
Primary Power	Triple 18650 Li-Ion Battery Holder	11.1 V Raw Bus	High-current motor & buck supply
Acoustic Feedback	Piezo Buzzer Module	A2 (Tone Output)	Audible alerts for 4 s calibration

3. CIRCUIT WORKING PRINCIPLE & SENSOR ACQUISITION

The line acquisition subsystem operates on differential infrared reflectance using the `QTRSensors` RC timing architecture. During each sampling cycle, the microcontroller drives pins `D9–D2` HIGH to charge the parallel node capacitors, switches them to high-impedance inputs, and measures the microsecond decay time for the voltage to fall below the logic threshold. Dark track surfaces absorb infrared radiation and yield prolonged discharge times, whereas white floor surfaces reflect strongly and discharge rapidly.

> **REVERSE-MOUNTED SENSOR PIN COMPENSATION & CENTROID FORMULA**
>
> Because the 8-channel sensor bar is physically mounted in a reversed orientation at the front of the chassis, the firmware maps the pins in descending order: `{9, 8, 7, 6, 5, 4, 3, 2}`. The function `qtr.readLineBlack(sensorValues)` normalizes each channel (0 to 1000) and computes a weighted centroid position P from **0** (extreme left) to **7000** (extreme right), with **3500** representing dead-center alignment: $\mathbf{P = [\ 0 \cdot S_0 + 1000 \cdot S_1 + 2000 \cdot S_2 + 3000 \cdot S_3 + 4000 \cdot S_4 + 5000 \cdot S_5 + 6000 \cdot S_6 + 7000 \cdot S_7\ ] / \sum S_i}$

4. HARDWARE ASSEMBLY & VISUAL DOCUMENTATION (PART I)

The physical construction of the autonomous robot was engineered around a black acrylic two-wheel-drive chassis utilizing modular solderless breadboards to facilitate rapid diagnostic probing and IC replacement. As shown in Figures 2 and 3, the front section houses the master rocker switch, status buzzer, and dual mini-breadboards (red for the Arduino Nano controller and white for the dual H-bridge motor driver IC).

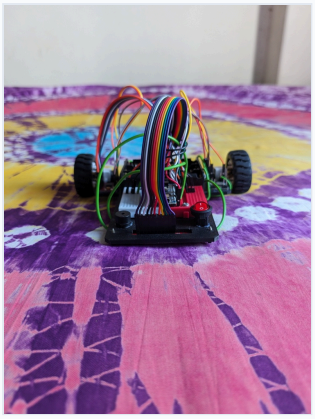


Figure 2: Front Perspective. Details the multi-wire rainbow ribbon harness interfacing the front-mounted 8-channel QTR-8RC sensor array with the central processing unit, flanked by the red master rocker switch and piezo buzzer.

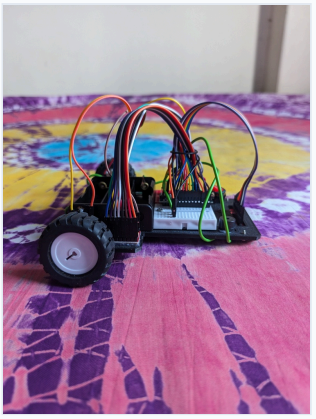


Figure 3: Right-Side Profile. Highlights the dual H-bridge motor driver DIP IC seated on the white mini-breadboard, the right-hand drive wheel assembly, and the sensor header termination.

4.1 Breadboard Partitioning & Signal Routing Architecture

Splitting the logic microcontroller and the power H-bridge onto separate adjacent mini-breadboards provides both mechanical modularity and electrical noise isolation. The rainbow ribbon cable routes the eight RC sensor lines vertically in an overarching service loop above the chassis deck. This high-clearance loop prevents magnetic coupling between the high-frequency PWM motor leads and the high-impedance RC discharge timing lines on pins `D2–D9`.

> **MECHANICAL KINEMATICS & DIFFERENTIAL STEERING GEOMETRY**
>
> The drive train employs two micro metal gearmotors mounted coaxially at the rear of the chassis via white polymer bracket blocks, coupled to high-grip treaded rubber tires. Because directional steering is achieved entirely through differential wheel velocities (`left` vs. `right` PWM duty cycles), keeping the heavy triple-18650 battery pack directly over the rear drive axle maximizes normal traction force and prevents wheel slippage during high-speed pivot maneuvers (`PIVOT_SPEED = 170`).

4. HARDWARE ASSEMBLY & VISUAL DOCUMENTATION (PART II)

Figures 4 and 5 illustrate the left-side voltage regulation stage and the rear power distribution bay. A critical hardware addition visible just ahead of the left wheel in Figure 4 is the dedicated step-down buck converter PCB mounted directly to the lower chassis plate.

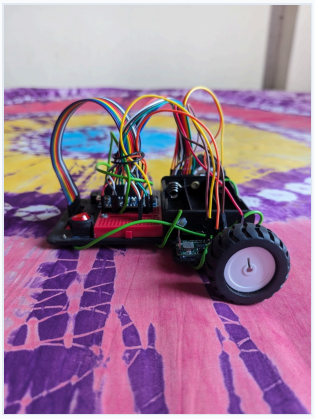


Figure 4: Left-Side Profile. Details the dedicated DC-DC step-down buck converter module mounted adjacent to the left drive wheel and the Arduino Nano seated on the red mini-breadboard.

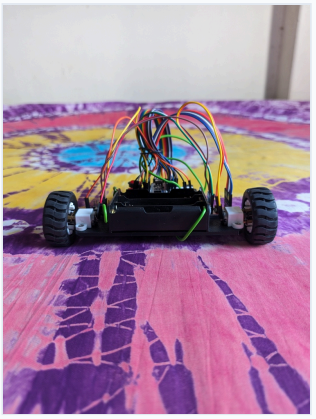


Figure 5: Rear Perspective. Shows the triple 18650 battery compartment, symmetrical white motor mounting brackets, rubber traction tires, and cross-chassis power distribution jumpers.

5. POWER DISTRIBUTION TROUBLESHOOTING & REGULATION ANALYSIS

> **FAILURE MODE ANALYSIS: INITIAL COMPONENT BURNOUT & INDUCTIVE TRANSIENTS**
>
> During initial prototype testing, feeding unregulated battery voltage directly into the logic ecosystem alongside active DC gearmotor switching resulted in catastrophic thermal and over-voltage failure of both the microcontroller and motor driver ICs. Commutation spikes and back-EMF generated during rapid motor deceleration coupled onto the logic supply rail, exceeding the absolute maximum ratings of the silicon gates.

To permanently eliminate rail instability before installing replacement chips, the power delivery network was re-architected and bench-tested. The raw series battery output (11.1 V nominal) was routed to feed the high-current motor driver supply rail directly, while a dedicated DC-DC step-down buck converter (Figure 4) was installed and calibrated on a benchtop supply to output a strictly regulated, ripple-filtered 5.0 V rail for the Arduino Nano and QTR-8RC sensor array. Sharing a single star-ground reference while decoupling the logic supply from motor current surges completely resolved IC overheating and reset brownouts.

6. CONTROL ALGORITHM & 7-ZONE BANG-BANG NAVIGATION

Rather than relying on a computationally heavy floating-point PID loop, the firmware implements a high-speed, 7-zone quantized Bang-Bang (piecewise-stepped proportional) controller executing at a 2 ms loop delay (~500 Hz update rate). At startup, the robot executes an automated 400-iteration sensor calibration sequence (~4 seconds) bracketed by 1200 Hz and 1800 Hz piezo buzzer tones. During navigation, the centroid reading `position` (0 to 7000) is partitioned into seven discrete steering zones that apply asymmetric PWM speeds to increase turning torque during sharp curves:

Zone Classification	Position Range (P)	Left PWM (PWMA)	Right PWM (PWMB)	Kinematic Action
1. Extreme Left	`P < 700`	0 (Stopped)	170 (`PIVOT_SPEED`)	Hard left pivot around locked inner wheel
2. Left Turn	`700 ≤ P < 1800`	40	150 (`SHARP_TURN_SPEED`)	Aggressive left corrective arc
3. Slight Left	`1800 ≤ P < 2800`	70	120 (`TURN_SPEED`)	Gentle left trim adjustment
4. Dead Center	`2800 ≤ P < 4200`	80 (`STRAIGHT_SPEED`)	80 (`STRAIGHT_SPEED`)	Balanced straight-line cruise
5. Slight Right	`4200 ≤ P < 5200`	120 (`TURN_SPEED`)	70	Gentle right trim adjustment
6. Right Turn	`5200 ≤ P < 6300`	150 (`SHARP_TURN_SPEED`)	40	Aggressive right corrective arc
7. Extreme Right	`P ≥ 6300`	170 (`PIVOT_SPEED`)	0 (Stopped)	Hard right pivot around locked inner wheel

6.1 Line Visibility Detection & State-Memory Recovery

On acute hairpins or track discontinuities, the robot may momentarily overshoot the black tape entirely. Before evaluating the zone ladder, the function `lineVisible()` scans all eight sensors to verify whether at least one channel exceeds a reflectance threshold of 200. Simultaneously, every valid loop updates a directional memory state variable (`lastDirection = -1` if `position < 3500`, else +1). If `lineVisible()` returns `false`, the controller immediately invokes `recover()`, executing a hard pivot at `PIVOT_SPEED (170)` toward the last known side until the line re-enters the sensor field.

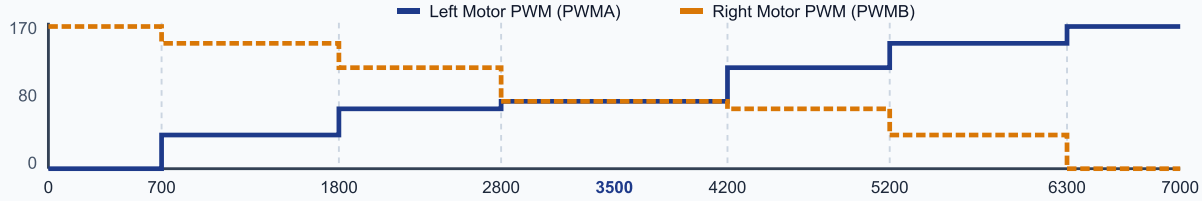


Figure 6: Piecewise-stepped PWM velocity profile for Left (solid blue) and Right (dashed amber) motors as a function of sensor centroid position *P*.

7. COMPLETE FIRMWARE IMPLEMENTATION (C++ / ARDUINO)

The complete C++ source code (`Bang_Bang_Final.ino`) deployed to the Arduino Nano microcontroller is presented below, encompassing pin definitions, sensor calibration, state-memory line recovery, and the 7-zone bang-bang loop:

```
#include <QTRSensors.h>
// ---------------- Motor Pins ----------------
#define PWMA 10
#define AIN1 12
#define AIN2 13
#define PWMB 11
#define BIN1 A0
#define BIN2 A1
#define BUZZER A2
#define LEDON A3

// ---------------- Speeds ----------------
#define STRAIGHT_SPEED 80
#define TURN_SPEED 120
#define SHARP_TURN_SPEED 150
#define PIVOT_SPEED 170
#define MAX_SPEED 200

// ---------------- Sensors ----------------
QTRSensors qtr;
const uint8_t SENSOR_COUNT = 8;
uint16_t sensorValues[SENSOR_COUNT];
int lastDirection = 1; // -1 = left, +1 = right

void setup() {
  Serial.begin(9600);
  pinMode(PWMA, OUTPUT); pinMode(AIN1, OUTPUT); pinMode(AIN2, OUTPUT);
  pinMode(PWMB, OUTPUT); pinMode(BIN1, OUTPUT); pinMode(BIN2, OUTPUT);
  pinMode(BUZZER, OUTPUT); pinMode(LEDON, OUTPUT);

  digitalWrite(LEDON, HIGH);
  qtr.setTypeRC();
  // Your sensor array is mounted backwards
  qtr.setSensorPins((const uint8_t[]){9,8,7,6,5,4,3,2}, SENSOR_COUNT);

  tone(BUZZER, 1200, 200); delay(300);
  for(int i = 0; i < 400; i++){
    qtr.calibrate();
    delay(10);
  }
  tone(BUZZER, 1800, 200); delay(300);
}
```

```
void moveForward(int left, int right){
  left = constrain(left, 0, MAX_SPEED);
  right = constrain(right, 0, MAX_SPEED);
  digitalWrite(AIN1, HIGH);
digitalWrite(AIN2, LOW);
  analogWrite(PWMA, left);
  digitalWrite(BIN1, HIGH);
digitalWrite(BIN2, LOW);
  analogWrite(PWMB, right);
}

bool lineVisible(){
  for(int i = 0; i < SENSOR_COUNT; i++){
    if(sensorValues[i] > 200) return true;
  }
  return false;
}

void recover(){
  if(lastDirection < 0) moveForward(0, PIVOT_SPEED);
  else moveForward(PIVOT_SPEED, 0);
}

void loop(){
  uint16_t position = qtr.readLineBlack(sensorValues);
  if(!lineVisible()){ recover(); return; }

  if(position < 3500) lastDirection = -1;
  else lastDirection = 1;

  // -------- Bang-Bang Zones --------
  if(position < 700)        moveForward(0, PIVOT_SPEED);
  else if(position < 1800) moveForward(40, SHARP_TURN_SPEED);
  else if(position < 2800) moveForward(70, TURN_SPEED);
  else if(position < 4200) moveForward(STRAIGHT_SPEED, STRAIGHT_SPEED);
  else if(position < 5200) moveForward(TURN_SPEED, 70);
  else if(position < 6300) moveForward(SHARP_TURN_SPEED, 40);
  else moveForward(PIVOT_SPEED, 0);

  Serial.print(position);
Serial.print('\t');
  Serial.print(sensorValues[0]);
Serial.print('\t');
  Serial.println(sensorValues[7]);
  delay(2);
}
```

FIRMWARE EXECUTION & TELEMETRY NOTES

The `Serial.print` diagnostic stream outputs the real-time centroid `position` alongside outer boundary sensors `sensorValues[0]` and `sensorValues[7]` at 9600 baud, allowing benchtop verification of the reverse-mounted pin array before untethered track runs.

8. DISCUSSION

The empirical performance of the autonomous robot across test tracks validates the effectiveness of quantized multi-zone bang-bang control for low-latency path tracking, while also highlighting important engineering trade-offs between discrete switching and continuous feedback control:

- **Discrete Multi-Zone Switching vs. Continuous PID Control:** Unlike a traditional Proportional-Integral-Derivative (PID) loop that computes smooth, continuous PWM adjustments, the 7-zone bang-bang architecture partitions the `0–7000` sensor span into discrete velocity steps. On sharp 90-degree turns and acute curves, this asymmetric stepping is highly advantageous: because the outer wheel accelerates up to `SHARP_TURN_SPEED (150)` or `PIVOT_SPEED (170)` while the inner wheel drops to `40` or `0`, the chassis overcomes static tire friction immediately without suffering from integral windup or derivative lag. Conversely, on long straightaways, transitioning across the boundary thresholds (`2800` and `4200`) introduces minor lateral oscillation ("hunting"), which is mitigated by keeping `STRAIGHT_SPEED` at a conservative duty cycle of `80`.
- **Power Rail Isolation & Hardware Reliability:** The most critical hardware lesson of this project emerged from the initial burnout of the microcontroller and motor driver ICs. Brushed DC gearmotors draw high stall currents during sudden pivot reversals and inject inductive back-EMF voltage spikes onto shared power lines. Introducing the dedicated step-down buck converter (Figure 4) to decouple the 5.0 V logic rail from the raw 18650 battery bus completely eliminated brownout resets and thermal IC failure.
- **Optical Calibration & State-Memory Recovery:** Because the `QTR-8RC` array measures capacitor decay times rather than fixed analog voltages, ambient infrared fluctuations can shift baseline readings. The automated 400-sample startup calibration routine (signaled by the piezo buzzer on `A2`) effectively normalizes each phototransistor against local surface contrast. Furthermore, the state-memory recovery routine (`lastDirection` paired with `lineVisible()`) proved essential whenever high entry speeds caused the front sensor bar to momentarily overshoot acute corners.

> **KEY ENGINEERING TAKEAWAY & FUTURE SCOPE**
>
> While solderless mini-breadboards allowed rapid re-wiring after the initial component failure, transitioning the final circuit onto a custom etched PCB with wide ground planes and adding a derivative (PD) damping term to the 7-zone velocity profile would allow higher straight-line cruise speeds (`PWM > 150`) without lateral chatter.

9. CONCLUSION

This personal engineering project successfully achieved the end-to-end design, hardware assembly, power-rail troubleshooting, and embedded C++ programming of an autonomous 2WD line-following robot. By diagnosing and resolving early catastrophic IC burnouts through the integration of a dedicated DC-DC step-down buck converter, a stable and noise-immune electrical architecture was established for the Arduino Nano and dual H-bridge driver.

On the software front, compensating for the reverse-mounted 8-channel `QTR-8RC` reflectance array in firmware and implementing an asymmetric 7-zone bang-bang control law enabled responsive trajectory tracking across both gentle arcs and sharp corners. Combined with the `lineVisible()` threshold detector and `lastDirection` pivot recovery mechanism, the robot demonstrated reliable, autonomous line-following capability under real-world operating conditions.